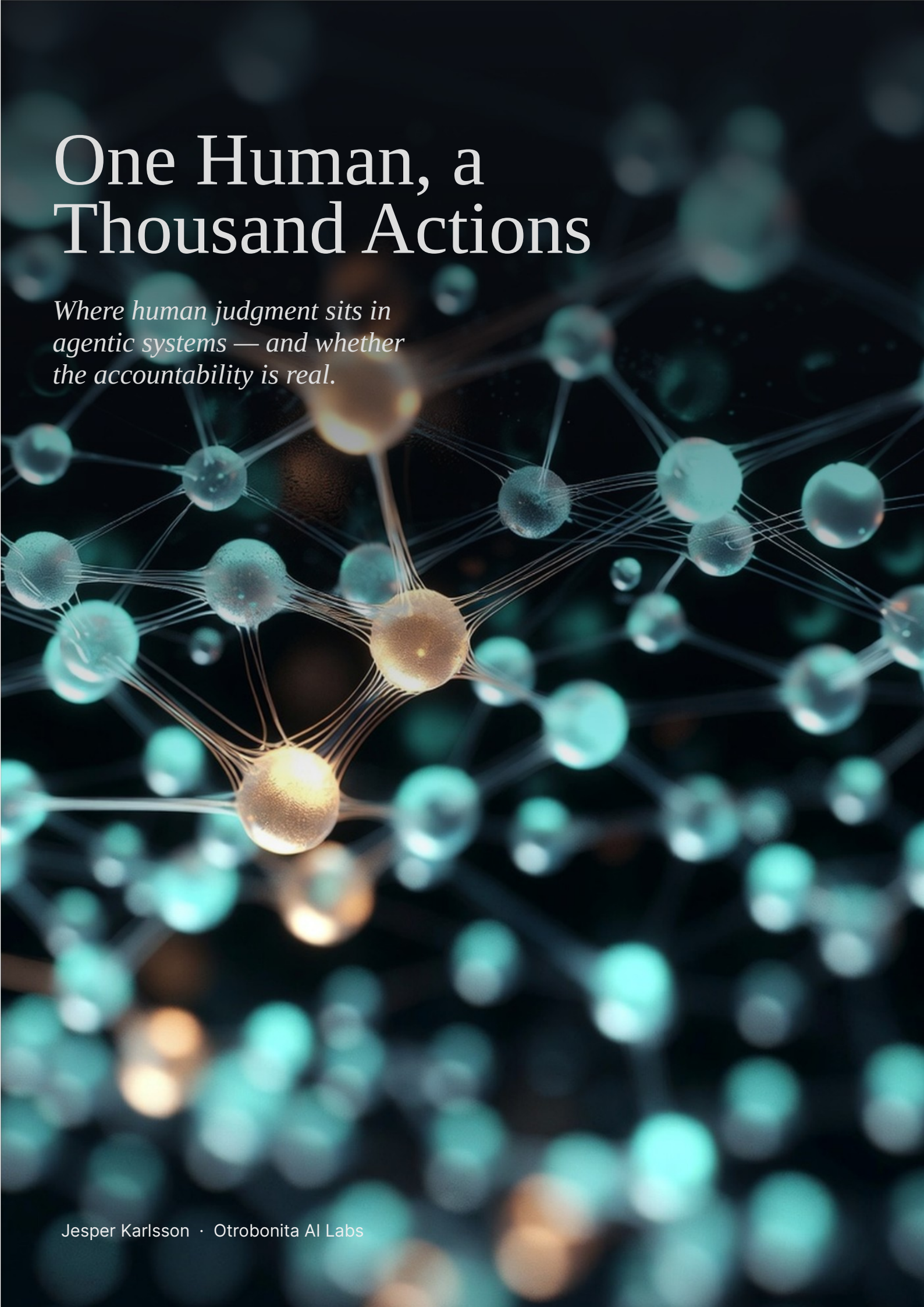
One Human, a Thousand Actions

Where human judgment sits in agentic systems — and whether the accountability is real.



Jesper Karlsson · Otrobonita AI Labs

If you read one page, read this one — the short version

Every major platform shift has produced the same pattern: engineers build the architecture, designers are invited to the demo. Mobile. Cloud. APIs. Each time, design got bolted on after the decisions were made. Agentic systems are moving faster than any of those shifts — and the pattern is already repeating. Orchestration graphs are being built. Touchpoints are being placed. Mostly by default.

Automation doesn’t reduce the need for design. It concentrates it.

When an agent handles a thousand routine actions, every moment a human appears carries the weight of all of them. Legal exposure, security boundaries, architectural commitments — compressed into one approval surface. A poorly designed form wastes a minute. A poorly designed approval at that scale is a governance failure. The value and risk at each judgment point will be larger than anything UX has been asked to carry before.

The design question has inverted.

Not what should the user see, but where should the human remain. Four answers, ascending: Human Approval (agent waits), Escalated to Human (agent calls), Human Notification (agent informs and continues), Autonomous (agent acts, every step reconstructable). Each is a designed choice. Made deliberately, autonomy earns trust. Made by default, it launders accountability.

The toolchain is reorganizing underneath us.

Behavior models, escalation rules, trust protocols — these are the new design deliverables. Experience logic still gets designed in Figma, but realized in frameworks like LangGraph, where approval checkpoints are native constructs. The gate you sketch maps to a node in production. Designers who understand that layer shape the system. Those who don’t get handed one already shaped.

Start where humans want out, not where AI looks impressive.

Consent is a feeling of retained control. Begin with work people are begging to be rid of — not work they care about owning. Each despised task absorbed builds the trust needed later, when agents reach decisions that actually matter.

Continuous systems raise every stake further.

A self-learning agent ships a starting model and rules for how it evolves. Touchpoints must lighten as trust accumulates; changes must be visible, not silent. At the limit, the agent proposes to change its own rules and a named human decides. Rubber-stamp decay there is a governance failure.

Where this doesn’t apply.

Where the human’s presence is the value — creative judgment, relationship-bearing conversations, decisions with legal or ethical weight. Removing the human there isn’t efficiency. It’s amputation.

Accountability needs somewhere to sit.

Naming a human as accountable for an agent's output is necessary and not sufficient. If the judgment points are in the wrong places — heavy where stakes are low, missing where they are high — that person is accountable in name only. Where the touchpoints sit determines whether the accountability is real.

Accountability without observability is nominal.

A human answerable for an agent's work has to reconstruct what happened between the gates — logs that record decisions rather than only events, traces that survive the hours between the work and the review, and access that does not require an engineer. Where the touchpoints sit decides whether accountability is real. Whether the human can see what happened decides whether it is possible at all.

> ***A hypothesis offered for argument — not a consulting offering, not a design methodology.***

What This Paper Is, and Isn't

This paper is offered as a point of view — a position to argue with, not a framework to implement. Its purpose is to make the current understanding of "UX work" feel less inevitable, and to give designers and the decision-makers they report to a concrete vocabulary for what changes when the systems they build stop waiting for input.

Everything that follows is deliberately specific. Where the paper sounds confident, read it as precision, not certainty. This is one possible account of design in the agentic era — not the account.

This is written for designers, but it is meant to be carried — into leadership meetings, portfolio discussions, and budget conversations. The changes it describes are not decisions a design team can make alone.

1. The Question Inverted

For thirty years, my profession has answered one question: what should the user see, and what should they do next? Screens, flows, affordances, feedback loops. However sophisticated the craft became, the underlying model held constant: the human operates, the interface responds.

Agentic AI breaks that model — not by making interfaces better, but by making the operator optional. An agent takes an intent — "monitor these defects overnight," "prepare the compliance summary" — determines the steps, selects the tools, and executes. The human doesn't operate anymore. The human delegates. And delegation is a fundamentally different design problem, because for long stretches of the workflow there is no user present at all.

The design question inverts. It is no longer what should the human see? It is where should the human remain? Every agentic workflow contains a finite number of moments where a human could see, approve, correct, redirect, or veto. Choosing those moments — their placement, frequency, weight — is now the core of the design job.

What “designing” means when there’s no screen to design

Visual and interaction craft do not disappear. Agents still need surfaces, and those surfaces deserve the same rigor as ever. What changes is what sits upstream. The primary design artifact is no longer the sequence of screens; it is the coordination model — the specification of when the system acts alone, when it reports, when it asks, and when it stops. The screens serve the model, not the other way around.

The shift from UX to autonomous experience does not make design less necessary. It makes bad design harder to see until it’s expensive — because a badly designed agentic flow doesn’t look ugly. It looks fine right up until trust collapses.

2. Trust Is the Deliverable

When a person delegates to an agent and looks away, the visible-trust mechanism is gone. Psychological distance creeps in. Users consistently report the same unease: what is it doing right now? How would I know if it went wrong? Trust, which used to be a byproduct of a well-crafted interface, becomes the primary deliverable.

Failure mode one: the confirmation-loop parody. The agent asks permission for everything. Autonomy is reduced to theater and the full cognitive load transfers back to the human, now with extra clicking. Users don’t feel in control; they feel like they’ve been made the clerk of a machine that was supposed to be their clerk.

Failure mode two: silent drift. The agent disappears and returns with something confidently, fluently wrong — having misread the intent somewhere around step four, with nobody watching. The output isn’t just wrong; it’s wrong with momentum. One experience like this costs more trust than fifty successes earn.

Researchers at Amazon describe workflows as coordination curves — how human involvement rises and falls across a task. The long valleys where the agent runs alone are exactly where design matters most. The related idea of responsive salience — an agent adjusting its visibility based on context and stakes — points at the same insight: visibility is now a designed, dynamic property.

A vocabulary of touchpoints

Four levels of weight, from heaviest to lightest:

Human Approval	Human must approve before agent continues	Consequences hard to reverse or carry external weight	Confirmation-loop parody
Escalated to Human	Agent detects problem or ambiguity and calls human	Agent recognises its own competence boundary	Alert fatigue; escalations ignored
Human Notification	Human is informed; agent continues	Awareness matters, intervention rarely does	Noise; filtered to a folder nobody opens
Autonomous	Agent acts alone — every step reconstructable after the fact	Routine work within established trust	Losing the audit trail destroys accountability

Two principles fall out. Weight must match stakes — an approval gate on a trivial action teaches users that gates are noise. And the taxonomy must be dynamic — a flow requiring Human Approval in month one should earn its way down to Notification by month six. Who decides when a touchpoint lightens? That is a real decision with real risk, and it should be owned deliberately — not drifted into by whoever last edited the configuration.

The asynchronous review problem

One property of agentic touchpoints has no precedent in screen design: the human may arrive at them hours after the work began. Rebuilding context to make an approval meaningful might take ten minutes — and under deadline pressure, the human will skip the rebuilding and approve on faith.

Every interrupting touchpoint therefore needs a context recovery surface: a compact answer to "what was I asked, what did the agent do, what changed, and what exactly am I approving?" A gate without context recovery is not oversight. It is a signature line.

3. The Toolchain Is Reorganizing

Designers used to hand off wireframes. Increasingly they hand off behavior models, escalation rules, trust protocols, and evaluation criteria. The deliverable is less "here is the screen" and more "here is when the system steps back and lets a human decide."

The experience logic still gets designed in Figma and its siblings. The blueprint just maps something new: not the user's path through screens, but the interleaving of human and agent activity. Which lane is the human in, and when?

That logic gets realized in orchestration frameworks — LangGraph being the clearest current example. Human-in-the-loop checkpoints are first-class constructs: an interrupt-and-wait-for-approval is native, configured per tool. Execution state is checkpointed during the pause, so the workflow resumes exactly where it stopped, whether the human answers in a minute or a day.

The escalation point you sketch in a design file has a direct counterpart in the orchestration code. This mapping is becoming real rather than already universal — the frameworks support it natively; the practice is still young. That is an argument for designers engaging with the layer now, not a claim that the pipeline is finished.

What this demands concretely: budget and time for designers to learn the orchestration layer; a seat in architecture discussions; a revision of what "design handoff" means in hiring profiles and review criteria. None of these are free. All are cheaper than retrofitting trust into a system that shipped without it.

4. Where to Start

Every organisation asks the same question: where do we begin with agentic flows? The instinct is to start with something visible and impressive. The instinct is wrong, for a specifically design reason: it ignores consent.

Consent is a feeling of retained control. Insert an agent into work people care about owning, and every touchpoint reads as surveillance. Insert an agent into work people despise, and the same touchpoints read as service. Start where the pain is sharpest and the consent is cheapest.

Time reporting may be the most universally hated activity in professional life. Nobody's identity is invested in it. An agent that makes it disappear triggers gratitude, not resistance. Expense claims, status-report compilation, meeting minutes: same category. There is empirical backing: in a recent enterprise study, 95% of participants required human confirmation before any critical business decision, and the top-ranked principle specified that agents should work autonomously only on low-impact, manually intensive tasks.

This on-ramp does three jobs: builds the trust budget for later; provides a low-stakes design lab to learn touchpoint craft; and surfaces the accountability question — who is answerable for the agent's output — at a scale where the answer is manageable.

What the first ninety days might look like

Pick one or two despised processes. Design the coordination model before writing orchestration code — every touchpoint, its weight, its context recovery surface. Instrument from day one. Define what "trust is building" looks like as a measurement, not a feeling. Review at week six and twelve — not for delivery metrics, but for whether the touchpoint weights are still right.

5. Where This Does Not Apply

Where engagement is the value. Creative direction, coaching, relationship-carrying conversations — where the human's presence is the product. Designing the human out isn't automation; it's amputation.

Where the law or ethics puts a human in the seat. Medical judgment, credit decisions, safety-critical operation. Here touchpoints are a compliance surface. A gate humans click through blind satisfies the letter and violates the point.

Where the workflow is genuinely novel every time. Early-stage strategy, crisis response — no repetition for trust to compound on. The human drives; agents are instruments.

Where users didn't choose the agent. Where an agent is imposed on people who bear its risks without sharing its benefits. The problem isn't placement. It's legitimacy.

6. Accountability Needs Somewhere to Sit

When execution stops being the scarce resource, judgment and accountability become the constraint. The governance answer that follows is familiar: name one human as accountable for each stream of agent-executed work, and have that person own a verification system rather than personally re-check everything.

That answer is necessary and incomplete. Naming someone accountable says who answers when something goes wrong. It says nothing about whether they had a genuine chance to prevent it. Within any given workflow the question recurs at a finer grain: where does that person's judgment

physically sit? If approval gates are too heavy where stakes are low, and absent where they are high, the accountable human is either rubber-stamping or discovering problems after they have shipped.

Touchpoint design is what makes an accountability model real rather than rhetorical. The org chart says who is answerable. The coordination model determines whether they were ever positioned to exercise the judgment they are answerable for. One is governance; the other is design; and the second is the load-bearing one.

What the accountable human needs to see

Accountability that cannot inspect is accountability in name only. A person answerable for an agent's output needs more than a gate in the right place; they need the means to reconstruct what happened between the gates — after the fact, on their own, without booking an engineer.

That resolves into three requirements, and none of them are infrastructure details to be settled later.

Logs that record decisions, not only events. A line stating that a tool was called is an operational record. The accountable human needs to know why the agent selected that tool, what it had concluded by that point, and what it ruled out. Systems that log outputs alone make the long autonomous stretches unreviewable by construction — the very stretches where nobody was watching.

Traces that survive the gap. A human arriving at an approval hours later needs the whole chain: the original intent, every step since, and what changed along the way. Distributed tracing already solves the equivalent problem between services. Agentic systems need the same discipline applied to reasoning steps — and the trace has to be legible to the person who is accountable, not only to the team that built the system.

Access the accountable person can actually exercise. If reconstructing a run takes a query language, a dashboard licence and somebody else's calendar, it will not happen under deadline. The human will approve on faith, and the record will show that a human approved. Reachability is a design requirement.

The touchpoint taxonomy already leans on this. *Autonomous* was defined as the agent acting alone with every step reconstructable after the fact, and that final clause carries the whole level. Without it, Autonomous is not a considered degree of trust. It is absent oversight wearing a label.

This is also where the responsibility genuinely sits with the people using the system. An organisation that accepts agentic output while declining to fund the means of inspecting it has not delegated work. It has delegated answerability, and kept the signature.

7. Designing Continuous & Self-Learning Agentic Systems

Most of what has been said so far applies to any agentic workflow. Continuous, self-learning systems raise the stakes further. When an agent keeps running and adjusting itself over weeks and months, the coordination model is no longer a static arrangement of touchpoints — it becomes a living system that must be allowed to evolve. Trust is no longer earned once; it is maintained or eroded across hundreds of small interactions.

Conventional product discovery is poorly equipped for this. It assumes a stable problem space, fixed user needs, and an interface that changes only through deliberate releases. Agentic systems that learn violate all three assumptions.

Why ordinary discovery is not enough

Traditional research asks what people need and how they currently work. That remains useful. It does not answer the questions that determine whether an agentic system will be accepted: What will people actually hand over? How much interruption will they tolerate? What proof do they require before acting on something a machine surfaced? How should the system's behaviour be allowed to change as trust accumulates?

Proposed additional discovery steps

- Coordination zone triage. Classify candidate workflows: done-with-me, done-for-me, or done-under-me. Only the first two require careful touchpoint design. Conventional discovery has no equivalent — conventional software offers only one zone.
- Delegation and anxiety mapping. Identify not only what people can delegate, but what they are willing to, and where anxiety concentrates. Autonomy that looks efficient on paper often fails when it touches work people feel defines their competence.
- Interrupt budget profile. Quantify interruption tolerance before placing a single gate. Fatigue is widely observed; it is almost never budgeted. This is one of the clearest practical disciplines this domain currently lacks.
- Trust baseline and verification audit. Establish what proof a given role needs before treating an agent's output as actionable. The gap between "the system produced an answer" and "I am prepared to act on it" is where many agentic initiatives die quietly.
- Evaluation pre-commitment. Define in advance what will be measured: intervention rate, escalation precision, time-to-detection, trust trajectory. Without this, teams measure whatever the system happens to emit.
- Coworker anchoring. In environments with co-determination traditions — and increasingly elsewhere — the affected workforce is a consultation partner. Monitoring boundaries, escalation rights, and conditions under which the system may change its own behaviour should be agreed, not announced. Systems that skip this are technically compliant and socially stillborn.

Designing for a system that is never finished

A continuous, self-learning agent ships a starting coordination model and a set of rules for how that model may evolve. Touchpoint weight should lighten as demonstrated reliability accumulates — visibly and explained. Labels and friction can shift with trust. The coordination curve itself can be shown over time, with a ghost of the original as comparison.

The governing principle: the system may change its behaviour, but it must not change silently. Opacity in the learning process destroys the very trust the adaptations are meant to serve.

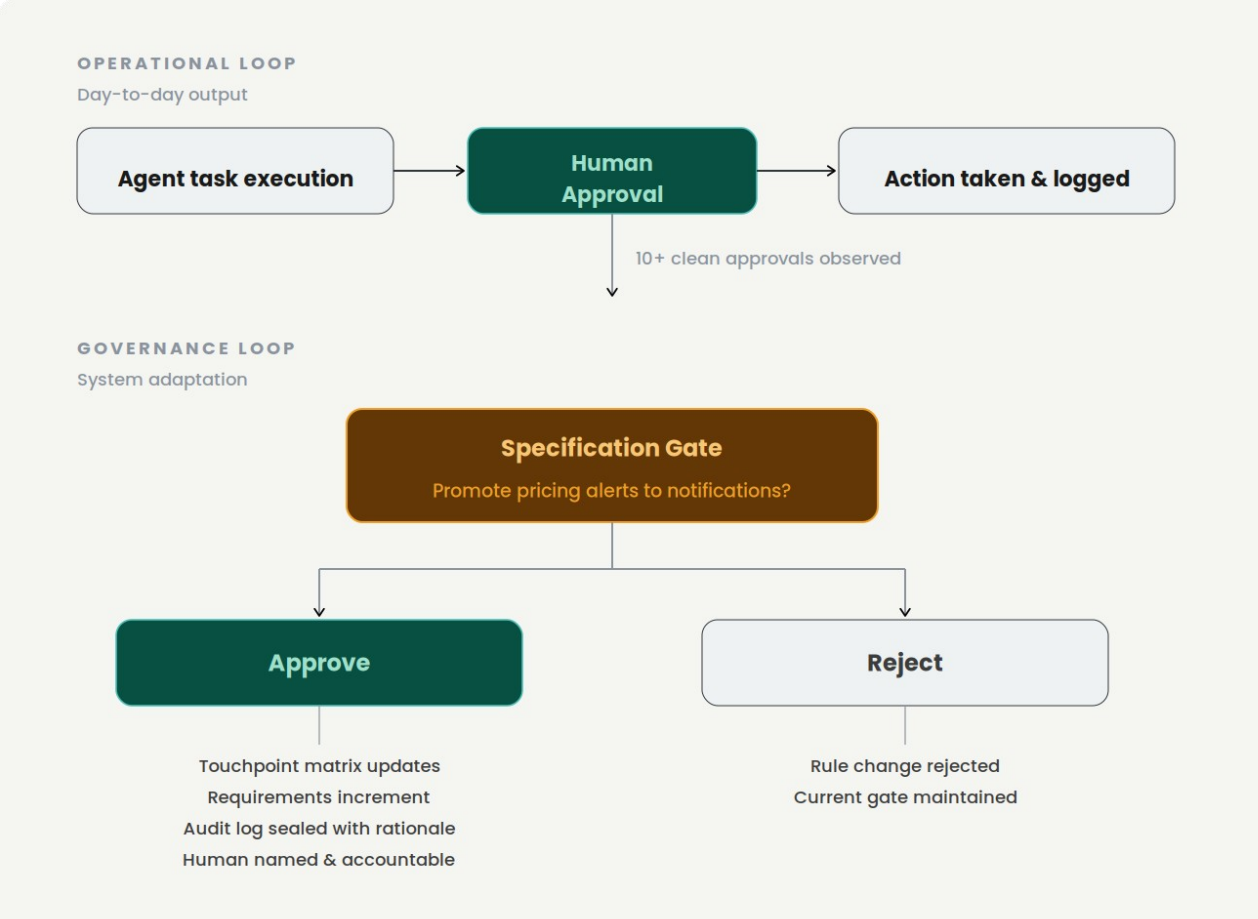


Figure 1 — The two loops. Operational touchpoints govern individual actions; a rule change governs the system's own behaviour.

When the agent proposes to change its own rules

Every touchpoint described so far governs a single action. A continuous, self-learning system introduces a qualitatively different kind: one where the agent proposes to change its own rules.

When the system observes that a category of alerts has been approved without modification fourteen times in a row, it may propose promoting them from Human Approval to Human Notification. That is not an operational decision. It is a governance decision — a change to the system's own specification. It belongs at its own moment, weighted deliberately and kept separate from the flow of ordinary approvals.

When the named accountable human — exercising exactly the judgment that accountability exists to locate — approves a rule change, all downstream artifacts change together: requirements increment, the touchpoint matrix updates, the interface specification reflects the new behaviour, and the audit log is sealed with the human's name, the decision, and the rationale.

Rubber-stamp decay on a rule change is not a quality problem. It is a governance failure. A human who clicks through without reading has not missed an error in an output. They have ceded control of the system's future behaviour — and remain, on the record, accountable for what that behaviour subsequently produces.

Rule changes should be infrequent by design, never batched, always accompanied by a plain-language summary of what changes and what the system observed to propose it, and always require an explicit rationale from the human approving them.

A note on ambition

The steps above are a proposal, not a proven method. They are offered because the alternative — forcing continuous agentic systems through discovery processes designed for static software — is visibly inadequate. Organisations that treat interrupt budgets, delegation anxiety, and evolving touchpoint weight as first-class design concerns will make fewer expensive mistakes.

8. What This Demands of Management

Touchpoint design must be budgeted as design work, not configuration. If the coordination model is an afterthought assigned to whoever writes the orchestration code, touchpoints will be placed by default. Defaults optimise for the demo, not for month six.

Designers need access to the orchestration layer — training time, tool budget, and a seat in architecture discussions. A small line item against the cost of retrofitting trust later.

The consent on-ramp requires patience that quarterly demos punish. Starting with time reporting produces no board-meeting fireworks. Leadership must measure the on-ramp by trust built, not headline impact.

Research budgets must follow the question. Researching trust in a system users don’t observe requires different methods and longer horizons than week-long usability studies.

The leadership decision agenda

Four decisions — not discussion topics — that should leave the meeting room owned and dated:

Who owns touchpoint evolution? Lightening a gate as trust accumulates is a risk decision. Name its owner, the criterion, and where it is recorded. For self-learning systems, this owner is the person who signs every rule change.

What is the rubber-stamp policy? Decide now what the organisation does when gate-approval times collapse below reading speed. Detection, response, and consequence, agreed before the first gate ships.

How is the consent on-ramp measured? Agree the trust-budget signals — intervention rates, touchpoint promotion rate, participant sentiment — so the on-ramp is judged on what it is for, not on delivery theatrics.

What is the minimum design investment in the orchestration layer? A number: training days, tool budget, whose calendar absorbs it. An unfunded intention is a decision to place touchpoints by default.

9. Open Questions

A paper earns trust by naming what it hasn’t answered.

The rubber-stamp decay. Every approval gate degrades. The hundredth approval gets a fraction of the attention the first did, and the gate silently converts from a judgment point into a latency tax — while still recording that a human approved, which is worse than no gate at all because it launders accountability. Two speculative patterns worth testing: Synthetic injection — periodically insert deliberate, low-risk errors and measure whether they are caught. Progressive friction — require the approver to identify what they are confirming rather than offering a single global button. Both carry costs; both are cheaper than discovering the gate had been decorative for months.

Researching the invisible. Our research methods evolved for observable interaction. How do you usability-test a valley in the coordination curve — a stretch where, by design, nothing is visible? Longitudinal trust measurement, incident-anchored interviews, and instrumented touchpoint analytics seem like pieces; none is yet a practice.

Metrics for a well-placed touchpoint. No standardised metrics for human-agent interaction quality yet exist. A provisional starter set: intervention rate at gates, escalation precision, time-to-detection when the agent errs, approval latency as a decay signal, and touchpoint promotion rate. Candidates, not standards.

The skill-transfer question. Interaction designers can become coordination designers. Some will. The craft draws on systems thinking and specifying behaviour rather than appearance — muscles adjacent to, not identical with, classical UX strengths. What the transition looks like for excellent visual craftspeople deserves more honesty than a single open question, but starts by being named.

Touchpoints across agent-to-agent chains. This paper assumes one human, one agent crew. Real systems chain agents across organisational boundaries. Where the human touchpoints belong when no single human owns the full chain is unresolved here, and it compounds the architectural-consistency problem that multi-stream agentic work already creates.

10. Closing Thought

The point of this paper is not that any particular touchpoint taxonomy is right. The point is that the question it forces — where should humans remain when systems no longer wait for them? — is coming regardless of what any design team decides. Every agentic system that ships is answering it, mostly by accident. And the self-learning systems now arriving answer it repeatedly, revising their own answer as they run.

The interface is receding; the judgment points remain; and they will be designed either deliberately or by default. In continuous systems, they will then be re-designed — by the system itself — under governance that is either legible or absent.

> ***Deliberately is a craft. It should be ours.***
>
> ***If this argument is wrong, it would be useful to know exactly where — that’s what it’s for.***

About This Paper

Jesper Karlsson is a UX and product designer, and founder of Otrobonita AI Labs. He has spent three decades in human–computer interaction, enterprise design systems and software development, most recently building autonomous agentic production systems.

Disagreement is the point. If part of this is wrong — and some of it will be — the author would rather hear it than not.

Cite as Karlsson, J. (2026). One Human, a Thousand Actions. Otrobonita AI Labs. otrobonita.com/whitepapers

Contact: jesper@otrobonita.com

The authoring team

- **Jesper Karlsson** — concept, thesis, ideas, structure, and accountability for every final call.
- **Fable 5, GPT 4.5, Grok 4.5, and Gemini 3.6** — drafting support, fact-checking, handling of uncertainty and counter-evidence, independent critique, and adversarial review.

Written in 2026.